

Table 1: Implementation of quantum computing in different domains

Domain	Implementation areas
Defence and strategic operations	Cryptanalysis operations, Simulation, Battle modelling, Communication security, Resource allocation, Strategy testing, Threat analysis and modelling, Radar enhancement, Secure communications, Simulation training
Materials science and chemistry	Material design, Nanostructures study, Crystal analysis, Magnetic properties, Molecule simulation, Drug design, Protein folding, Reaction prediction, Catalyst discovery, Thermal conductivity
Artificial intelligence, Machine learning, Deep learning	Quantum classifiers, Pattern recognition, Feature extraction, Data clustering, Neural networks, Object recognition, Natural language processing (NLP)
Energy, oil and gas	Battery optimisation, Grid management, Solar simulation, Oil exploration, Fuel efficiency, Reservoir simulation, Risk assessment, Pipeline monitoring, Drilling optimisation, Refinery control
Space exploration and aerospace	Flight simulation, Materials testing, Satellite routing, Trajectory optimisation, Sensor fusion, Orbit simulation, Resource analysis, Rover navigation, Planet mapping, Asteroid tracking
Sensors and climatic analysis	Quantum imaging, Weather modelling, Energy forecasting, Ocean simulation, Emission tracking, Disaster prediction, Temperature measurement, Chemical detection, Magnetic sensing, Pressure monitoring
Healthcare and pharmaceuticals	Drug discovery, Disease simulation, Protein modelling, Genome analysis, Diagnostic tools, Molecular docking, Disease modelling, Drug testing, Trial simulation, Compound screening
Manufacturing	Process optimisation, Quality assurance, Robotics control, Defect detection, Materials testing
Automotive	Traffic optimisation, Crash simulation, System testing, Path planning, Sensor fusion, Emission analysis, Sensor analysis, EV battery, Autonomous navigation, Vehicle design
Telecommunications	Signal processing, Quantum repeaters, Error correction, Network routing, Spectrum allocation
Agriculture and food	Crop prediction, Yield optimisation, Pest management, Soil analysis, Irrigation planning, Recipe optimisation, Nutrition analysis, Quality control, Supply prediction, Preservation modelling
Quantum internet	Entanglement distribution, Network scaling, Node routing, Secure channels, Error correction
Education	Quantum teaching, Student assessment, Algorithm training, Lab simulations, Curriculum design
Smart cities and environment	Traffic modelling, Pollution monitoring, Climate modelling, Water quality, Habitat simulation, Resource management, Sensor networks, Waste management, Energy optimisation, Public safety
Entertainment and gaming	Level optimisation, Multiplayer synchronisation, Physics simulation, AI opponents, Movie rendering, Content generation, Virtual reality (VR) simulation, Game physics, Sound optimisation, Procedural generation

- quantum hashing, code breaking, secure protocols
- **Insurance:** Customer analysis, risk evaluation, claim prediction, fraud detection, policy pricing
- **Logistics:** Route optimisation, traffic modelling, inventory management, fleet scheduling, supply prediction
- **Retail:** Demand forecasting, customer segmentation, inventory planning, pricing analysis, recommendation systems

A number of free and open source libraries and frameworks are available for implementing the algorithms and circuits of quantum machine learning. Most of these are compatible with Python programming. Table 2 lists some of these.

Variational quantum classification using PennyLane on a fintech application

The following example uses a sample dataset of

cryptocurrencies. Using QML, the aim is to predict if the closing value of the cryptocurrency the next day will be higher than its price today. The quantum circuit of 1 qubit is taken with a small variational block.

To install PennyLane, type:

```
$ pip install pennylane
```

The quantum machine learning-based binary classification is as follows:

```
import pandas as pd
from pennylane import numpy as np
import pennylane as qml
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
```